

Chapter 8. Board configuration

Board configuration is the process of customising the SDK to run on a specific board design. The SDK comes with some predefined configurations for boards produced by Raspberry Pi and other manufacturers, the main (and default) example being the Raspberry Pi Pico.

Configurations specify a number of parameters that could vary between hardware designs. For example, default UART ports, on-board LED locations and flash capacities etc.

This chapter will go through where these configurations files are, how to make changes and set parameters, and how to build your SDK using CMake with your customisations.

8.1. The Configuration files

Board specific configuration files are stored in the SDK source tree, at `.../src/boards/include/boards/<boardname>.h`. The default configuration file is that for the Raspberry Pi Pico, and at the time of writing is:

`<sdk_path>/src/boards/include/boards/pico.h`

This relatively short file contains overrides from default of a small number of parameters used by the SDK when building code.

SDK: <https://github.com/raspberrypi/pico-sdk/blob/master/src/boards/include/boards/pico.h>

```

1  /*
2  * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3  *
4  * SPDX-License-Identifier: BSD-3-Clause
5  */
6
7  // -----
8  // NOTE: THIS HEADER IS ALSO INCLUDED BY ASSEMBLER SO
9  //      SHOULD ONLY CONSIST OF PREPROCESSOR DIRECTIVES
10 // -----
11
12 // This header may be included by other board headers as "boards/pico.h"
13
14 // pico_cmake_set PICO_PLATFORM=rp2040
15
16 #ifndef _BOARDS_PICO_H
17 #define _BOARDS_PICO_H
18
19 // For board detection
20 #define RASPBERRYPI_PICO
21
22 // --- UART ---
23 #ifndef PICO_DEFAULT_UART
24 #define PICO_DEFAULT_UART 0
25 #endif
26 #ifndef PICO_DEFAULT_UART_TX_PIN
27 #define PICO_DEFAULT_UART_TX_PIN 0
28 #endif
29 #ifndef PICO_DEFAULT_UART_RX_PIN
30 #define PICO_DEFAULT_UART_RX_PIN 1
31 #endif
32
33 // --- LED ---
34 #ifndef PICO_DEFAULT_LED_PIN

```

```

35 #define PICO_DEFAULT_LED_PIN 25
36 #endif
37 // no PICO_DEFAULT_WS2812_PIN
38
39 // --- I2C ---
40 #ifndef PICO_DEFAULT_I2C
41 #define PICO_DEFAULT_I2C 0
42 #endif
43 #ifndef PICO_DEFAULT_I2C_SDA_PIN
44 #define PICO_DEFAULT_I2C_SDA_PIN 4
45 #endif
46 #ifndef PICO_DEFAULT_I2C_SCL_PIN
47 #define PICO_DEFAULT_I2C_SCL_PIN 5
48 #endif
49
50 // --- SPI ---
51 #ifndef PICO_DEFAULT_SPI
52 #define PICO_DEFAULT_SPI 0
53 #endif
54 #ifndef PICO_DEFAULT_SPI_SCK_PIN
55 #define PICO_DEFAULT_SPI_SCK_PIN 18
56 #endif
57 #ifndef PICO_DEFAULT_SPI_TX_PIN
58 #define PICO_DEFAULT_SPI_TX_PIN 19
59 #endif
60 #ifndef PICO_DEFAULT_SPI_RX_PIN
61 #define PICO_DEFAULT_SPI_RX_PIN 16
62 #endif
63 #ifndef PICO_DEFAULT_SPI_CSN_PIN
64 #define PICO_DEFAULT_SPI_CSN_PIN 17
65 #endif
66
67 // --- FLASH ---
68
69 #define PICO_BOOT_STAGE2_CHOOSE_W25Q080 1
70
71 #ifndef PICO_FLASH_SPI_CLKDIV
72 #define PICO_FLASH_SPI_CLKDIV 2
73 #endif
74
75 // pico_cmake_set_default PICO_FLASH_SIZE_BYTES = (2 * 1024 * 1024)
76 #ifndef PICO_FLASH_SIZE_BYTES
77 #define PICO_FLASH_SIZE_BYTES (2 * 1024 * 1024)
78 #endif
79 // Drive high to force power supply into PWM mode (lower ripple on 3V3 at light loads)
80 #define PICO_SMPS_MODE_PIN 23
81
82 #ifndef PICO_RP2040_B0_SUPPORTED
83 #define PICO_RP2040_B0_SUPPORTED 1
84 #endif
85
86 // The GPIO Pin used to read VBUS to determine if the device is battery powered.
87 #ifndef PICO_VBUS_PIN
88 #define PICO_VBUS_PIN 24
89 #endif
90
91 // The GPIO Pin used to monitor VSYS. Typically you would use this with ADC.
92 // There is an example in adc/read_vsys in pico-examples.
93 #ifndef PICO_VSYS_PIN
94 #define PICO_VSYS_PIN 29
95 #endif
96
97 #endif

```

As can be seen, it sets up the default UART to `uart0`, the GPIO pins to be used for that UART, the GPIO pin used for the on-board LED, and the flash size.

To create your own configuration file, create a file in the board `../source/folder` with the name of your board, for example, `myboard.h`. Enter your board specific parameters in this file.

8.2. Building applications with a custom board configuration

The CMake system is what specifies which board configuration is going to be used.

To create a new build based on a new board configuration (we will use the `myboard` example from the previous section) first create a new build folder under your project folder. For our example we will use the `pico-examples` folder.

```
$ cd pico-examples
$ mkdir myboard_build
$ cd myboard_build
```

then run cmake as follows:

```
$ cmake -D"PICO_BOARD=myboard" ..
```

This will set up the system ready to build so you can simply type `make` in the `myboard_build` folder and the examples will be built for your new board configuration.

8.3. Available configuration parameters

[Table 34](#) lists all the available configuration parameters available within the SDK. You can set any configuration variable in a board configuration header file, however the convention is to limit that to configuration items directly affected by the board design (e.g. pins, clock frequencies etc.) Other configuration items should generally be overridden in the CMake configuration (or another configuration header) for the application being built.